

Exercícios: Revisão de Fundamentos

Turma: LionsDev

Tópicos: Revisão dos fundamentos de programação.

1. Sistema de Cobrança Básico (Variáveis, I/O e Operadores)

Um restaurante precisa automatizar o cálculo de suas contas. Peça ao garçom para registrar o valor total consumido na mesa e a quantidade de pessoas que irão dividir a conta.

Antes de calcular, valide se a quantidade de pessoas informada é maior que zero. Caso não seja, exiba "Erro: A quantidade de pessoas deve ser maior que zero" e encerre. Caso seja válido, calcule o valor da taxa de serviço (10% sobre o consumo) e some ao valor total. Em seguida, divida esse novo total pela quantidade de pessoas. Exiba no terminal um resumo mostrando o valor da taxa, o valor total com a taxa e quanto cada pessoa deverá pagar (formatado como moeda).

2. Catraca de Acesso (Estruturas de Seleção e Lógica)

Um evento corporativo possui regras rígidas de acesso. Peça ao usuário para informar a sua idade e pergunte: "Você possui credencial VIP? (sim/nao)".

Avalie o cenário: caso a pessoa seja menor de 18 anos, exiba "Acesso Negado: Menor de idade". Caso tenha 18 anos ou mais E possua a credencial VIP, exiba "Acesso Liberado para a Área VIP". Para qualquer outra situação (maior de idade sem VIP), exiba "Acesso Liberado para a Pista Comum".

3. Máquina de Café Automática (Seleção de Casos)

Simule o painel de uma máquina de bebidas. Exiba um menu numérico com quatro opções: 1 - Café Expresso, 2 - Cappuccino, 3 - Chá, 4 - Chocolate Quente.

Peça ao usuário para digitar o código numérico da bebida desejada. Utilize uma estrutura de seleção de casos para avaliar o código e exiba a mensagem "Preparando seu [Nome da Bebida]...". Caso o usuário digite um número fora do menu, exiba "Código inválido. Selecione uma opção existente."

4. Painel de Login com Tentativas (Laços de Repetição)

Um sistema corporativo possui um login com senha padrão (`"admin2024"`). O usuário tem no máximo 5 tentativas para acertar.

Crie uma rotina com um laço de repetição que solicite a senha ao usuário. A cada tentativa incorreta, exiba "Senha incorreta. Tentativas restantes: X" (mostrando quantas restam). Se o usuário acertar a senha, exiba "Acesso Autorizado!" e encerre. Se esgotar as 5 tentativas sem acertar, exiba "Conta Bloqueada: Número máximo de tentativas excedido" e encerre o programa. Ao final, exiba quantas tentativas foram utilizadas.

5. Balanço Diário de Vendas (Arrays e Laços)

O gerente de uma loja precisa saber o total faturado no dia. Crie um array vazio chamado `vendasDoDia`.

Peça ao gerente para digitar o valor de 5 vendas realizadas, uma de cada vez, e insira cada valor no array. Após capturar todas, crie uma rotina de repetição que percorra essa lista, some todos os valores e descubra o faturamento total. Após o laço, exiba o total faturado e calcule a média de valor por venda realizada.

6. Ficha de Colaborador (Objetos e Condicionais)

Um departamento de RH precisa atualizar o cadastro de um funcionário. Crie um objeto contendo o nome, o cargo e o salário atual.

Avalie os dados desse registro: caso o salário seja inferior a R\$ 2.500,00, aplique um aumento de 15%. Caso seja igual ou superior, aplique apenas 5%. Adicione uma nova propriedade ao objeto chamada `revisaoAplicada` com o valor `true`. Imprima o registro atualizado no console para conferência.

7. Conversor de Medidas Logísticas (Funções Tradicionais)

Uma transportadora internacional precisa converter pesos de libras (lb) para quilogramas (kg). Crie uma função tradicional que receba um valor em libras, faça o cálculo (`libras / 2.2046`) e devolva o resultado em kg.

No programa principal, peça o peso em libras ao usuário. Caso o valor informado seja negativo, exiba "Valor inválido". Caso seja positivo, acione a função e exiba o resultado retornado com duas casas decimais.

8. Filtro de Qualidade de Peças (Arrow Function e Arrays)

Uma fábrica analisa o tamanho de peças produzidas. A tolerância de qualidade exige que o tamanho esteja entre 48 e 52 (inclusive). Crie uma arrow function que receba o tamanho de uma peça e retorne `true` caso esteja dentro dessa faixa, e `false` caso contrário.

Crie um array vazio chamado `pecasAprovadas`. Peça ao inspetor para digitar o tamanho de 5 peças diferentes. Para cada peça, use a sua arrow function; se o retorno for verdadeiro, insira o

tamanho no array de aprovadas. Ao final, exiba a lista de aprovadas e a quantidade total de peças que passaram no teste.

9. Frota de Entregas (Objetos e Arrays Internos)

Uma empresa possui um veículo com as propriedades: `placa` e `quilometragens` (um array com as distâncias percorridas nos últimos 3 dias).

Crie uma rotina que percorra o array de quilometragens dentro do objeto e some a distância total. Caso a soma passe de 500 km, adicione uma propriedade `manutencao: true` ao objeto; caso contrário, `manutencao: false`. Exiba o objeto completo e o total de km percorridos.

10. O Terminal do Gerente (O Desafio Final)

Vamos criar um mini sistema de gestão financeira. Crie um objeto `"ContaEmpresarial"` com as propriedades: `titular`, `saldo` (inicie em zero) e `historico` (um array vazio).

Utilize um laço `do-while` para garantir que o menu seja exibido ao menos uma vez. O menu deve oferecer as opções:

1. **Depositar:** Peça o valor, some ao saldo e adicione um objeto ao histórico: `{ tipo: "Depósito", valor: X }`.
2. **Sacar:** Peça o valor. Se houver saldo, subtraia e adicione ao histórico: `{ tipo: "Saque", valor: X }`. Caso contrário, avise "Saldo Insuficiente".
3. **Extrato:** Percorra o array de histórico e exiba cada operação realizada.
4. **Sair:** Encerre o programa.

Dica: o `prompt-sync` devolve os dados como texto (string). Para fazer contas ou comparar maior/menor, converta a entrada para número antes de usar.